

DIAGNÓSTICO DE INGENIERÍA DE SOFTWARE I

Ingeniería de Software y SDLC

De la necesidad al software operable

PRINCIPIOS

MÉTODOS

PROCESOS

HERRAMIENTAS

PRD

SRS

ARQUITECTURA

RELEASE PLAN

¿Qué es Ingeniería de Software?

DEFINICIÓN

Disciplina para construir y evolucionar software de forma sistemática

INTEGRA:

PRINCIPIOS

MÉTODOS

PROCESOS

HERRAMIENTAS

BUSCA:

CALIDAD

PREVISIBILIDAD

MANTENIBILIDAD

REDUCCIÓN DEL RIESGO

Los 4 componentes

1 Principios

Reglas para tomar buenas decisiones cuando no hay una receta exacta

2 Métodos

Formas estructuradas de organizar el trabajo (Scrum, Cascada, Kanban)

3 Procesos

Actividades concretas, con responsables, entradas y salidas

4 Herramientas

Instrumentos que soportan el trabajo: IDE, CI/CD, tableros

¿Qué es el SDLC?

DEFINICIÓN

Ciclo para transformar una necesidad en software y mantenerlo

SIGNIFICA:

SOFTWARE DEVELOPMENT LIFE CYCLE



Enfoque lineal

A veces, con requisitos muy estables



Iterativo, con retroalimentación

Lo más frecuente en la práctica

Etapas del SDLC



1. Planificación y viabilidad



1

Planificación

1 **Problema y objetivo** qué dolor real estamos resolviendo

2 Stakeholders y alcance inicial de la primera versión

3 Viabilidad técnica, económica y operativa

4 Riesgos, restricciones y criterios de éxito

2. Requisitos: PRD + SRS



2

Requisitos

1 **PRD** qué producto necesita el negocio/usuario y por qué

2 **SRS** requisitos del sistema y comportamiento esperado

3 Criterios de aceptación y trazabilidad

4 **Gate** requisitos revisados antes de implementar

3. Diseño: arquitectura



3

Diseño

1 **Diseño de solución** componentes, interfaces y datos

2 Decisiones técnicas y su justificación

3 Diagrama de arquitectura obligatorio como evidencia

4 **Gate** arquitectura coherente con PRD/SRS

Release Plan: el camino de implementación

RP

ARTEFACTO CLAVE

Release Plan

1

Ordena entregas e incrementos del proyecto

2

Define qué entra en cada release y cuándo

3

Dependencias, prioridades y criterios de salida

4

Gate PRD + SRS + Arquitectura + Release Plan

El mapa antes de programar



4–7. Implementación, pruebas, despliegue y operación



4. Implementación

Código + integración + revisiones (code review)

5. Pruebas

Verificar que se cumplen requisitos y calidad

6. Despliegue

Llevar el incremento a un entorno real

7. Operación

Monitorear, mantener y aprender del sistema

Principios aplicados a un proyecto real

1 KISS

Busquen la solución más simple que funcione, no la más ingeniosa

2 DRY

No se repitan: si copian y pegan tres veces, necesitan una función

3 SOLID

Responsabilidades claras: cada módulo, una razón para cambiar

4 Acoplamiento y cohesión

Módulos independientes entre sí y consistentes por dentro

5 Seguridad desde el diseño

No es un parche final, se integra en la arquitectura

6 Automatización y trazabilidad

Automatizar pruebas/despliegues; registrar decisiones

Métodos de desarrollo: comparación

1 Cascada

Requisitos estables y alta previsibilidad

2 Iterativo / Incremental

Entregar por partes y aprender en el camino

3 Prototipado

Validar UX o reducir incertidumbre

4 Scrum

Backlog priorizado y cadencias fijas (sprints)

5 Kanban

Flujo continuo con límites de trabajo en curso (WIP)

6 DevOps

Integra desarrollo, despliegue y operación

7 TDD / BDD

Pruebas y comportamiento esperado junto al código

¿Cuándo usar cada enfoque?

Situación	Enfoque recomendado
Alta regulación / requisitos estables	Planificado o híbrido
Producto nuevo / alta incertidumbre	Agile + prototipos
Entregas frecuentes	Incremental + CI/CD + DevOps
Soporte y flujo continuo	Kanban
Calidad crítica	TDD/BDD + automatización
Contextos complejos	Híbrido, con gates explícitos

Proceso típico y roles

1 Product Manager / Owner

Valor y prioridades: qué se construye primero

2 Business Analyst

Traduce necesidades del negocio en requisitos

3 Software Architect

Decisiones técnicas estructurales

4 UX/UI

Experiencia e interfaz del producto

5 Developers

Implementación del sistema

6 QA / Test Engineer

Verificación de requisitos y calidad

7 DevOps / SRE

Entrega continua y operación

8 Stakeholders

Validación y decisiones de negocio

Gates de control



Calidad y trazabilidad

Requisito → Diseño → Código → Prueba → Release

1

Métricas de calidad, defectos, cobertura y rendimiento

2

Revisión humana de decisiones críticas

3

Gestión explícita de cambios y desviaciones

Síntesis para recordar



Ingeniería de Software $\neq$ solo programar



SDLC = ciclo completo del software



Método = cómo trabajamos



Proceso = qué hacemos y quién lo hace



**PRD + SRS + Arquitectura + Release Plan =
el mapa antes de implementar**



**Calidad = responsabilidad durante todo el
ciclo**

Actividad 2: de la teoría a la práctica

1 Redactar el PRD

Alcance inicial de requerimientos:
problema, objetivo y criterios de éxito

2 Redactar el SRS

Historias de usuario con criterios de
aceptación en formato Gherkin

3 Clasificar requisitos

Requisitos funcionales, no funcionales y
de sostenibilidad

4 Trazar y validar

Matriz de trazabilidad y evidencia de
validación con el stakeholder

Cómo redactar un PRD: alcance inicial de requerimientos

- 1 **Problema y contexto** qué dolor real resuelve el producto y para quién
- 2 **Objetivo del producto** qué resultado medible se espera lograr
- 3 **Usuarios y stakeholders** quiénes lo usan y quiénes deciden sobre él
- 4 **Alcance inicial** qué incluye esta primera versión y qué queda explícitamente afuera
- 5 **Restricciones y supuestos** tiempo, presupuesto, tecnología y regulaciones
- 6 **Criterios de éxito** cómo se sabrá que el producto cumplió su objetivo

Cómo redactar un SRS: historias de usuario y Gherkin

HISTORIA DE USUARIO

Como [rol],
quiero [funcionalidad],
para [beneficio].

EJEMPLO

Como estudiante, quiero recuperar mi contraseña por correo, para volver a acceder a mi cuenta sin contactar a soporte.

CRITERIO DE ACEPTACIÓN (GHERKIN)

Dado [contexto inicial],
Cuando [acción del usuario],
Entonces [resultado esperado].

EJEMPLO

Dado que el estudiante olvidó su contraseña, cuando solicita la recuperación con su correo registrado, entonces el sistema envía un enlace válido por 30 minutos.

Cada historia de usuario debe trazarse a uno o más requisitos (RF/RNF) del SRS y a al menos un criterio de aceptación verificable.

Tipos de requisitos: RF, RNF y sostenibilidad

RF

Requisitos Funcionales

Qué debe hacer el sistema: funciones, comportamientos y reglas de negocio observables.

- El sistema debe permitir iniciar sesión con correo y contraseña.
- El sistema debe generar un comprobante al confirmar un pedido.
- El sistema debe notificar el cambio de estado de una solicitud.

RNF

Requisitos No Funcionales

Cómo debe comportarse el sistema: calidad, rendimiento y restricciones técnicas.

- Tiempo de respuesta menor a 2 segundos por transacción.
- Disponibilidad del servicio de al menos 99.5% mensual.
- Los datos personales se cifran en tránsito y en reposo.

SOS

Requisitos de Sostenibilidad

Impacto ambiental, social y económico del sistema a mediano y largo plazo.

- Optimizar el cómputo para reducir el consumo energético del servidor.
- Garantizar accesibilidad para personas con discapacidad (WCAG).
- Diseñar para bajo costo de mantenimiento y vida útil extendida.

Matriz de trazabilidad: formato esperado

Formato mínimo esperado — pueden ampliar columnas según la complejidad de su proyecto:

Requisito	Historia de usuario	Criterio de aceptación	Evidencia / Prueba
RF-01	Como usuario quiero iniciar sesión con mi correo, para acceder a mis datos.	Dado credenciales válidas, cuando inicio sesión, entonces accedo al sistema.	Caso de prueba TC-01 + captura de pantalla
RNF-02	Como usuario quiero que el sistema responda rápido, para no perder tiempo.	Dado 100 usuarios concurrentes, cuando consulto, entonces responde en menos de 2s.	Prueba de carga / reporte de rendimiento
SOS-01	Como institución quiero reducir el consumo de recursos, para bajar el impacto ambiental.	Dado el reporte mensual de uso, cuando se optimiza el proceso, entonces el consumo de CPU baja 10%.	Métrica de monitoreo antes / después

Entregables de la Actividad 2

- 1 **PRD completo** alcance inicial de requerimientos, objetivo y criterios de éxito
- 2 **SRS con historias de usuario** formato Como/Quiero/Para, con criterios de aceptación en Gherkin
- 3 **Clasificación explícita de requisitos** RF, RNF y sostenibilidad en cada historia
- 4 **Matriz de trazabilidad** cada requisito conectado a su historia, criterio y evidencia
- 5 **Evidencia de validación con el stakeholder** acta, correo o formulario que confirme su aprobación

Preguntas de repaso

?

¿Cuál es la diferencia entre PRD y SRS?

?

¿Qué problema resuelve el Release Plan?

?

¿Qué significa aplicar KISS, DRY y SOLID?

?

¿Por qué el diagrama de arquitectura debe presentarse en Diseño?

?

¿Cuándo conviene Scrum, Kanban o Cascada?

?

¿Qué roles participan y qué responsabilidad tiene cada uno?